Neuromorphic ASR
Transducer Models on Memristor Hardware

Azim Javed
azim.javed@ml.rwth-aachen.de

October 15, 2026

Advisor: Benedikt Hilmes and Simon Berger
Supervisor: Priv.-Doz. Ralf Schlüter

Human Language Technology and Pattern Recognition
Computer Science Department, RWTH Aachen University

Outline

Related Works

Motivation

Prerequisites

Experiments and Results

Future Work

Related Works

- **Monotonic Recurrent Neural Network Transducer (RNN-T) and Decoding Strategies** [Tripathi$^+$ 19]
  - Introduces monotonic alignments for RNN-T
- **Conformer: Convolution-Augmented Transformer for Speech Recognition** [Gulati$^+$ 20]
  - Introduces the Conformer encoder
- **Synaptogen: A Cross-Domain Generative Device Model for Large-Scale Neuromorphic Circuit Design** [Hennen$^+$ 24]
  - Synaptogen: Memristor device model for a family of devices
- **SynaptogenML** [Rossenbach$^+$ 25]
  - Extends Synaptogen and introduces SynaptogenML

Outline

Related Works

Motivation

Prerequisites

Experiments and Results

Future Work

Motivation

- Memristor hardware enables fast, energy-efficient neural network inference.
  → but introduces analog inaccuracies.
- CTC-based Conformers have been evaluated on memristor hardware [Rossenbach+ 25].
- Transducers add two components, which are repeatedly invoked at inference:
  - Predictor network
  - Joiner network
- Objective:
  - Characterize the effect of quantization and device noise on transducer ASR.
  - Assess training and decoding countermeasures.

Outline

Related Works

Motivation

Prerequisites

Experiments and Results

Future Work

Overview

- A memristor is an analog computing device with a programmable resistance state [Strukov+ 08].
- Enables fast, energy-efficient implementation of Vector-Matrix Multiplication (VMM)

  → and thus neural networks.
- Limitations:
  - Introduces analog inaccuracies.
  - Production is still in early stages.
  - Experiments are simulated using SynaptogenML [Rossenbach+ 25].

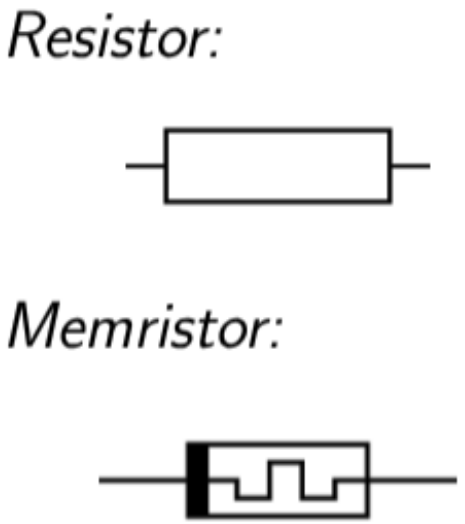


Figure: Resistor and memristor schematics

Analog In-Memory Computing

- Multiplication computation via Ohm's law:

$$\boldsymbol{V = IR} \tag{1}$$

- Voltage $\boldsymbol{V}$, current $\boldsymbol{I}$, and resistance $\boldsymbol{R}$.
- Possible to rewrite resistance as conductance $\boldsymbol{G}$:

$$\boldsymbol{I = GV} \quad \text{with} \quad \boldsymbol{G = \frac{1}{R}} \tag{2}$$

- Addition done via Kirchhoff's law:
  - For any connected node in a circuit, with $\boldsymbol{N}$ connected lines:

$$\sum_{n=0}^{N} \boldsymbol{I_n = 0} \;\rightarrow\; \boldsymbol{I_0 = -\sum_{n=1}^{N} I_n} \tag{3}$$

- Currents can be trivially added by simply connecting electric lines.

Analog In-Memory Computing

- Resistors as the connecting element between vertical and horizontal lines.
- Each resistor's conductance is an element of a constant matrix.
- Column sum via line connection.

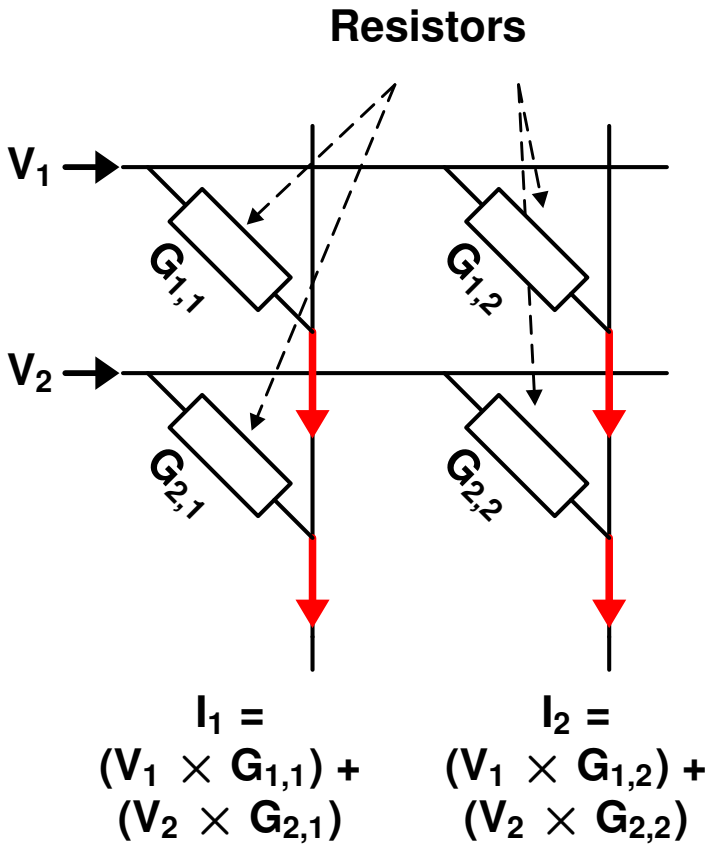


Figure: Crossbar array performing analog VMM [Rossenbach+ 25]

The Memristor

Key Idea: Encode each weight bit as a high- or low-resistance state.

- Let V_S be the set voltage and V_R the reset voltage thresholds.
- Apply $V < V_S \rightarrow$ set to low resistance.
- Apply $V > V_R \rightarrow$ set to high resistance.
- Intermediate states are difficult to program reliably for the devices measured in Synaptogen.
- Memristor state is non-volatile.

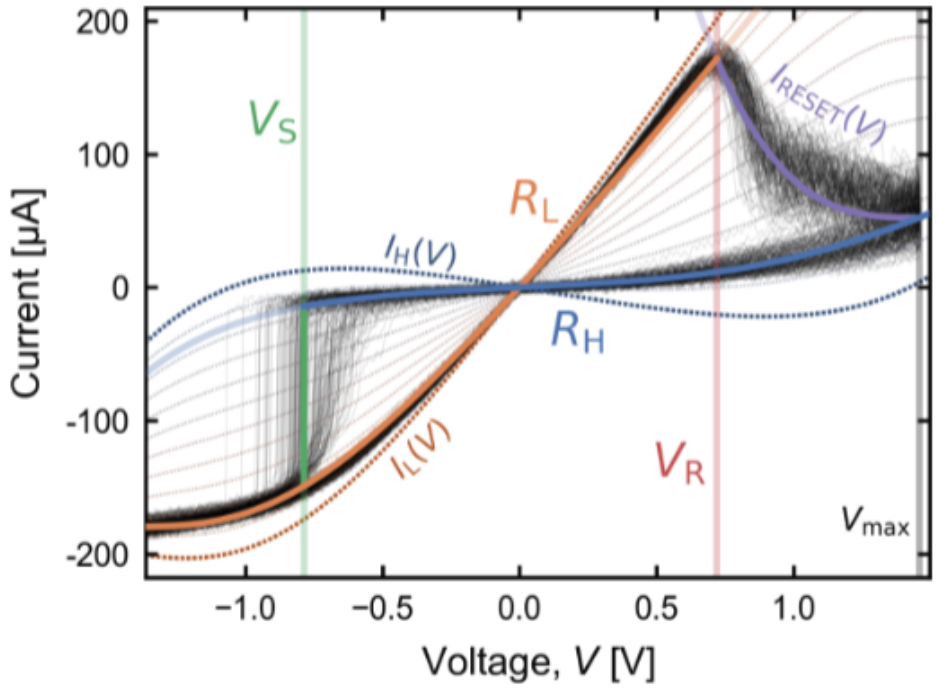


Figure: I–V characteristic curve of a memristor [Hennen+ 24]

R_L/R_H: low-/high-resistance state; $I_L(V)/I_H(V)$: corresponding current; $V_{\max}$: maximum applied voltage; $\mathrm{RESET}(V)$: current during the reset transition.

Limitations So Far

- Resulting resistance state varies with each programming cycle.
- Non-linear relation of current and voltage.

- Only positive, non-zero weights possible.
- Current leakage prohibits a true zero.

Memristor Crossbar Layout

- Use positive and negative bitlines to create memristor pairs
- Each row-wise pair of memristors represents a weight

→ For N input rows, with row index i and output-column index j:

$$I_j = \sum_{i=1}^{N} V_i G_{i,j}^{+} - \sum_{i=1}^{N} V_i G_{i,j}^{-}$$

Benefits and limitations:

- Pairs reduce the zero-weight offset of a single bitline.
- Exact conductance matching is not guaranteed.

→ Nominal zero weights can retain a residual difference.

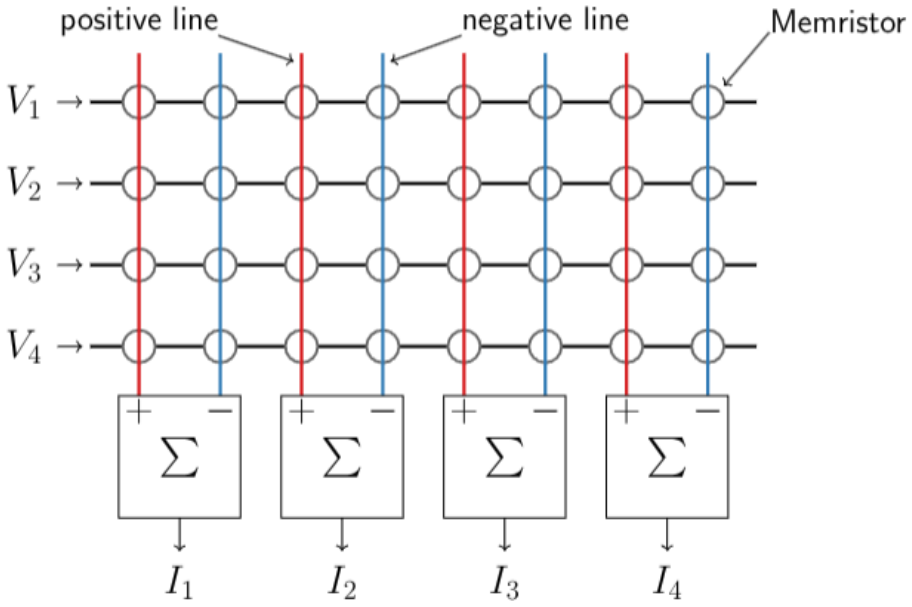


Figure: Paired memristor array structure [Aguirre+ 24]

Outline

Related Works

Motivation

Prerequisites

Experiments and Results

Future Work

Transducer Deployment Scenarios

- Which components of the model can be deployed on the device?
  - ▷ Encoder only vs. encoder + predictor vs. encoder + predictor + joiner.
- Tradeoff: more offloading ⇒ more energy efficiency, but also more noise.
- Optional external language model during decoding.

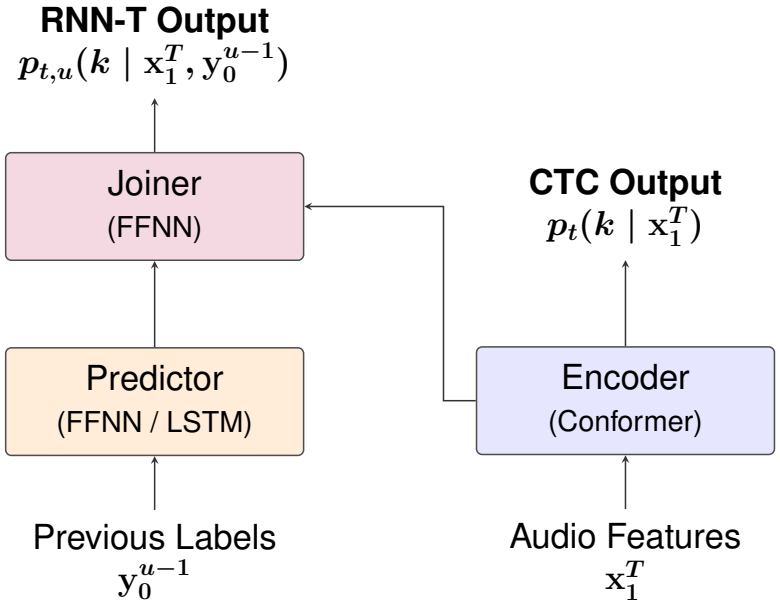


Figure: CTC vs. Transducer architecture components

FFNN: feed-forward neural network; **LSTM**: long short-term memory; **LM**: language model.

k: candidate label or blank; t: acoustic-frame index; u: output-label index.

$\mathrm{x}_1^T = (\mathrm{x}_1, \ldots, \mathrm{x}_T)$: audio-feature sequence; $\mathrm{y}_0^{u-1} = (y_0, \ldots, y_{u-1})$: previously emitted labels;

Experimental Setup

- Data: LibriSpeech [Panayotov+ 15], 960 h.
- Targets: 184 Byte-Pair Encoding (BPE) subword labels.
- Features: 80 log-mel filters, 10 ms frame shift; SpecAugment during training.
- Encoder: 12-layer Conformer [Gulati+ 20], 512-dim.
- Temporal downsampling: factor TBD.
- Training epochs:
  - Full-precision baseline: TBD.
  - QAT from scratch: TBD; QAT fine-tuning: TBD.
- Model sizes (million parameters): CTC TBD; FFNN transducer TBD; full-context TBD.

Experimental Setup: Models and Quantization

- Model types:
  1. Connectionist Temporal Classification (CTC) [Graves$^+$ 06]: encoder only.
  2. FFNN transducer: 2-layer, 640-dim feed-forward predictor.
     - Conditions on the previous-label embedding.
  3. Full-context transducer: LSTM predictor over label history.
- Both transducers use monotonic alignments [Tripathi$^+$ 19].
- Quantization: Quantization-Aware Training (QAT).
- Per-tensor symmetric; 8-/4-bit weights, 8-bit activations; zero-point 0.
- Training and evaluation:
  - Full-precision baseline and QAT training from scratch.
  - Evaluate QAT models with memristor simulation (3–5 runs).

CTC Results

Table: WER (%; mean ± standard deviation) on LibriSpeech dev-other – CTC. Memristor results use 5 simulation runs.

		Lex-Free	Tree	
Evaluation	**Weight bits**	**No LM**	**No LM**	**4-gram**
Baseline	FP32	7.1	6.5	5.2
QAT	8	7.3	6.8	5.3
QAT	4	8.3	7.6	5.7
Memristor simulation	8	13.5 ± 0.37	10.4 ± 0.20	6.7 ± 0.09
Memristor simulation	4	11.9 ± 0.18	10.1 ± 0.12	6.8 ± 0.12

FFNN Transducer Results: 8-bit Quantization

Table: WER (%; memristor mean ± standard deviation) on LibriSpeech dev-other: 8-bit quantization vs. baseline.

Modules Quantized			Lex-Free	Tree	
Encoder	**Predictor**	**Joiner**	**No LM**	**No LM**	**4-gram**
Baseline					
–	–	–	5.9	5.8	4.8
QAT					
✓	–	–	6.3	6.1	4.9
✓	✓	–	6.4	6.2	5.0
✓	✓	✓	6.4	6.2	4.9
Memristor Simulation					
✓	–	–	9.8 ± 0.06	8.7 ± 0.03	6.7 ± 0.07
✓	✓	–	10.7 ± 0.10	8.9 ± 0.11	6.7 ± 0.04
✓	✓	✓	44.7 ± 1.95	56.1 ± 10.90	18.8 ± 1.24

Updated full-model 4-gram aggregate; run counts are not supplied in the report.

FFNN Transducer Results: 4-bit Quantization

Table: WER (%; memristor mean ± standard deviation) on LibriSpeech dev-other: 4-bit quantization vs. baseline.

Modules Quantized			Lex-Free	Tree	
Encoder	**Predictor**	**Joiner**	**No LM**	**No LM**	**4-gram**
Baseline					
–	–	–	5.9	5.8	4.8
QAT					
✓	–	–	7.2	6.8	5.2
✓	✓	–	7.3	6.9	TBD
✓	✓	✓	TBD	TBD	TBD
Memristor Simulation					
✓	–	–	10.1 ± 0.04	9.1 ± 0.12	6.6 ± 0.07
✓	✓	–	10.4 ± 0.11	9.3 ± 0.12	6.7 ± 0.06
✓	✓	✓	37.4 ± 1.33	29.6 ± 2.39	12.8 ± 0.72

Full-model 4-gram: ILM 0.4, ELM 0.5 (best reported setting). TBD: QAT-only scores not supplied.

FFNN Transducer: Error Breakdown

Table: LibriSpeech dev-other, lexicon-free search without LM; encoder-only quantization and device simulation. Error rates as percentages of reference words.

Evaluation	Weight bits	Substitutions (%)	Insertions (%)	Deletions (%)
Baseline	FP32	TBD	TBD	TBD
QAT	8	TBD	TBD	TBD
QAT	4	TBD	TBD	TBD
Memristor simulation	8	TBD	TBD	TBD
Memristor simulation	4	TBD	TBD	TBD

TBD: fill in the error rates; average device-simulation rates over the same runs as the WER tables.

FFNN Transducer: Fine-Tuning vs. QAT from Scratch

Table: WER (%; memristor mean ± standard deviation) on LibriSpeech dev-other: QAT fine-tuning from a full-precision checkpoint vs. full pre-training (PTN). FFNN transducer with encoder under QAT.

(a) QAT

		Lex-Free	Tree	
Training	**Weight bits**	**No LM**	**No LM**	**4-gram**
Fine-tuned	8	6.6	6.3	5.0
Full PTN	8	6.5	6.2	5.1
Fine-tuned	4	7.2	7.0	5.1
Full PTN	4	7.2	6.8	5.2

(b) Memristor Simulation

		Lex-Free	Tree	
Training	**Weight bits**	**No LM**	**No LM**	**4-gram**
Fine-tuned	8	8.1 ± 0.06	7.5 ± 0.07	5.8 ± 0.08
Full PTN	8	9.8 ± 0.06	8.7 ± 0.03	6.7 ± 0.07
Fine-tuned	4	13.1 ± 0.06	11.2 ± 0.11	7.2 ± 0.05
Full PTN	4	10.1 ± 0.04	9.1 ± 0.12	6.6 ± 0.07

FFNN Transducer: Internal LM Compensation

Table: WER (%; mean ± standard deviation) on LibriSpeech dev-other (Tree + 4-gram LM search). Memristor results use 3 simulation runs.

(a) Fixed ELM scale = 0.6

	Modules Quantized		ILM Scale				
	Encoder	**Predictor**	0.1	0.2	0.3	0.4	0.5
Baseline	–	–	**4.8**	**4.8**	**4.8**	5.0	5.1
QAT	✓	–	**5.1**	**5.1**	**5.1**	5.3	5.4
	✓	✓	**5.0**	**5.0**	5.1	5.2	5.4
Memristor	✓	–	17.4 ± 1.60	8.5 ± 0.34	7.0 ± 0.06	6.8 ± 0.08	**6.7 ± 0.10**
	✓	✓	18.7 ± 1.58	8.8 ± 0.23	7.2 ± 0.09	**6.8 ± 0.03**	6.9 ± 0.10

(b) Optimal LM Scales

	Modules Quantized		Scales		
	Encoder	**Predictor**	**ILM**	**ELM**	**WER (%)**
Baseline	–	–	0.1	0.6	4.8
QAT	✓	–	0.2	0.7	5.0
	✓	✓	0.2	0.7	5.0
Memristor	✓	–	0.3	0.5	6.7 ± 0.07
	✓	✓	0.4	0.5	6.7 ± 0.04

ILM: Internal Language Model; **ELM**: External Language Model.

Full-Context Transducer Results: 8-bit Quantization

Table: WER (%; memristor mean ± standard deviation) on LibriSpeech dev-other: 8-bit quantization vs. baseline.

Modules Quantized			Lex-Free	Tree	
Encoder	**Predictor**	**Joiner**	**No LM**	**No LM**	**4-gram**
Baseline					
–	–	–	5.8	5.6	4.9
QAT					
✓	–	–	5.9	5.7	5.2
✓	✓	–	6.1	5.9	5.2
✓	✓	✓	6.1	5.9	5.2
Memristor Simulation					
✓	–	–	9.0 ± 0.29	8.1 ± 0.16	7.1 ± 0.07
✓	✓	–	12.8 ± 0.65	10.7 ± 0.84	8.0 ± 0.28
✓	✓	✓	29.5 ± 1.14	39.5 ± 1.13	TBD

Encoder+predictor uses cached prediction; 4-gram: ILM 0.5, ELM 0.5 (best reported setting).

Outline

Related Works

Motivation

Prerequisites

Experiments and Results

Future Work

Future Work

- Fine-tuning:
  - Fine-tuning from a full-precision checkpoint for other configurations
- Error Analysis:
  - Substitutions vs. insertions vs. deletions.

  → How can we improve the degradations in the lexicon-free search?
  - Study acoustic posterior distortion under noise.
- Generalization:
  - Evaluation and training on larger/out-of-domain datasets.

Future Work: Search

- Decoder Parameters:
  - Tune emission-score penalties and biases (e.g., for blank emissions).
  - Tune softmax temperature to sharpen or soften the distribution under noise.
- Search Space vs. Model & QAT Level:
  - Larger search spaces require more hypothesis expansions and energy.
  - Study the impact of restricting the search space on WER and Real-Time Factor (RTF).

  → How many more hypotheses are needed for on-device (simulated) vs. 4-bit vs. 8-bit vs. full-precision runs?

  → Compare hypothesis expansion across CTC and transducer variants.
- Search Paradigms:
  - Explore alternative search algorithms and language model combinations (e.g., lexicon-free with LSTM/transformer-based LM).

Future Work: Robust Training & Countermeasures

- Noise-and-Quantization Aware Training:
  - Inject simulated crossbar conductance noise directly into training forward pass.
- Selective Layer Mapping:
  - Deal with noise in noise-sensitive components (e.g., joint network).

  → Mixed-precision quantization
- Denoising & Output Compensation:
  - Post-processing to correct analog inaccuracies.

  → Train a small neural network to denoise the memristor output.

  → Use a small calibration dataset to learn a linear transformation to correct the output.

Thank you for your attention!

References

- F. Aguirre, A. Sebastian, M. Le Gallo, W. Song, T. Wang, J. J. Yang, W. D. Lu, M.-F. Chang, D. Ielmini, Y. Yang, A. Mehonic, A. J. Kenyon, M. A. Villena, J. B. Roldán, Y. Wu, H.-H. Hsu, N. Raghavan, J. Suñé, E. Miranda, A. Eltawil, G. Setti, K. Smagulova, K. N. Salama, O. Krestinskaya, X. Yan, K.-W. Ang, S. Jain, S. Li, O. Alharbi, S. Pazos, M. Lanza.
  Hardware Implementation of Memristor-Based Artificial Neural Networks.
  Nature Communications, Vol. 15, No. 1, pp. 1974, March 2024.
- A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, K. Keutzer.
  A Survey of Quantization Methods for Efficient Neural Network Inference, Aug. 2021.
  arXiv:2103.13630.
- A. Graves, S. Fernández, F. Gomez, J. Schmidhuber.
  Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks.
  In *Proc. ICML*, pp. 369–376, Pittsburgh, PA, June 2006.

- A. Gulati, J. Qin, C.-C. Chiu, N. Parmar, Y. Zhang, J. Yu, W. Han, S. Wang, Z. Zhang, Y. Wu, R. Pang.
  Conformer: Convolution-Augmented Transformer for Speech Recognition.
  In *Proc. Interspeech*, pp. 5036–5040, Shanghai, China, Oct. 2020.
- T. Hennen, L. Brackmann, T. Ziegler, S. Siegel, S. Menzel, R. Waser, D. J. Wouters, D. Bedau.
  Synaptogen: A Cross-Domain Generative Device Model for Large-Scale Neuromorphic Circuit Design.
  IEEE Transactions on Electron Devices, Vol. 71, No. 9, pp. 5345–5353, Sept. 2024.
  arXiv:2404.06344.
- V. Panayotov, G. Chen, D. Povey, S. Khudanpur.
  Librispeech: an ASR Corpus Based on Public Domain Audio Books.
  In *Proc. IEEE ICASSP*, pp. 5206–5210, Queensland, Australia, April 2015.

- N. Rossenbach, B. Hilmes, L. Brackmann, M. Gunz, R. Schlüter.
  Running Conventional Automatic Speech Recognition on Memristor Hardware: A Simulated Approach.
  In *Proc. Interspeech*, Rotterdam, The Netherlands, Aug. 2025. arXiv:2505.24721.
- D. B. Strukov, G. Snider, D. R. Stewart, R. S. Williams.
  The missing memristor found.
  Nature, Vol. 453, pp. 80–83, 2008.
- A. Tripathi, H. Lu, H. Sak, H. Soltau.
  Monotonic Recurrent Neural Network Transducer and Decoding Strategies.
  In *Proc. IEEE ASRU*, pp. 944–948, Sentosa, Singapore, Dec. 2019.

In-Memory Computing [Rossenbach+ 25]

- Various technologies exist, with varying tradeoffs:
  - Chip complexity and density
  - Energy consumption
  - Speed and reliability
- This talk focuses on memristive hardware, in particular resistive RAM (ReRAM):
  - Key benefits: low energy consumption, non-volatility

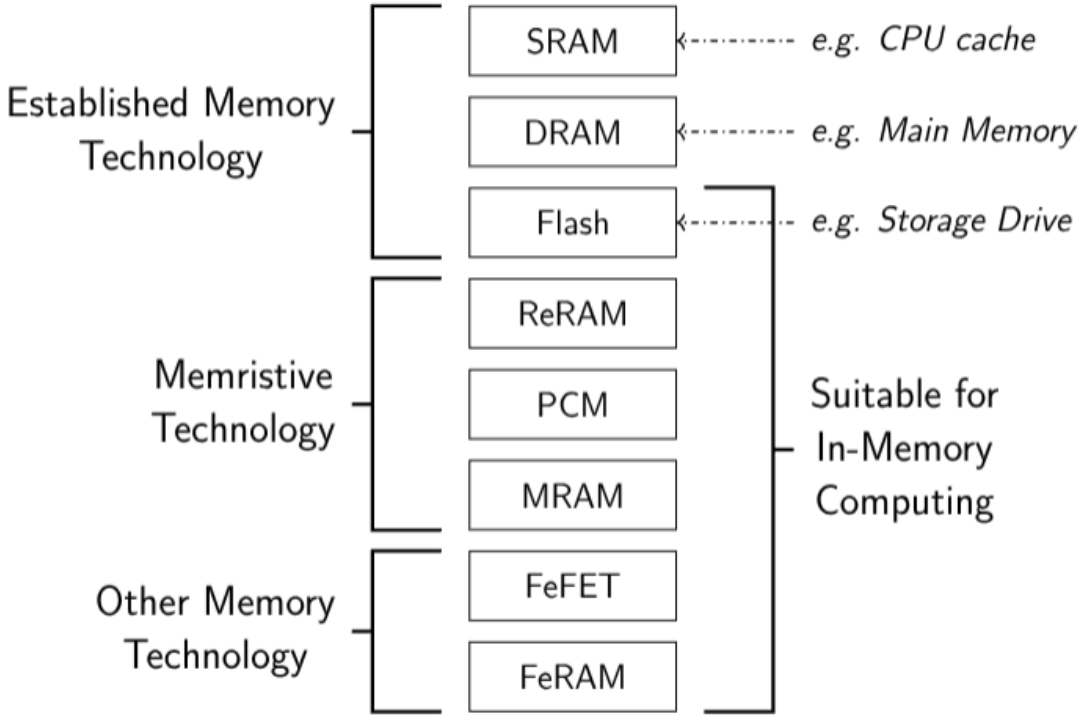


Figure: Taxonomy of In-Memory Computing technologies

Quantization

- The number of memristors required is proportional to the precision of the weights.
  - Need to reduce the precision of the weights → quantization.
- Quantization-Aware Training (QAT) performs better than Post-Training Quantization (PTQ), but requires more training time [Gholami+ 21].

→ High performance gap at low precision

Table: WER (%; mean ± standard deviation) on TED-LIUMv2 dev, Conformer CTC, 4-gram Language Model (LM) [Rossenbach+ 25]

Weight bits	FP32	8	6	5	4	3	2
PTQ	7.2	7.2	7.9	8.9	11.4	30.7	–
QAT		7.3	7.7	7.4	7.8	8.3	22.1

Memristor Neural Layers [Rossenbach$^+$ 25]

- Tile the weight matrix:
  - Tile weight matrix with arbitrary dimensions into smaller fixed-size arrays.
  - E.g., 512×512 weight matrix $\rightarrow 16$ crossbar arrays of size 128×128.
- Slice weights into memristor states:
  - Binary setting (high vs. low resistance).
  - Stacking of memristors for each magnitude bit (k-bit weight requires $k - 1$ arrays).
- Pair positive $(+)$ and negative $(-)$ bitlines.

Memristor Neural Layers

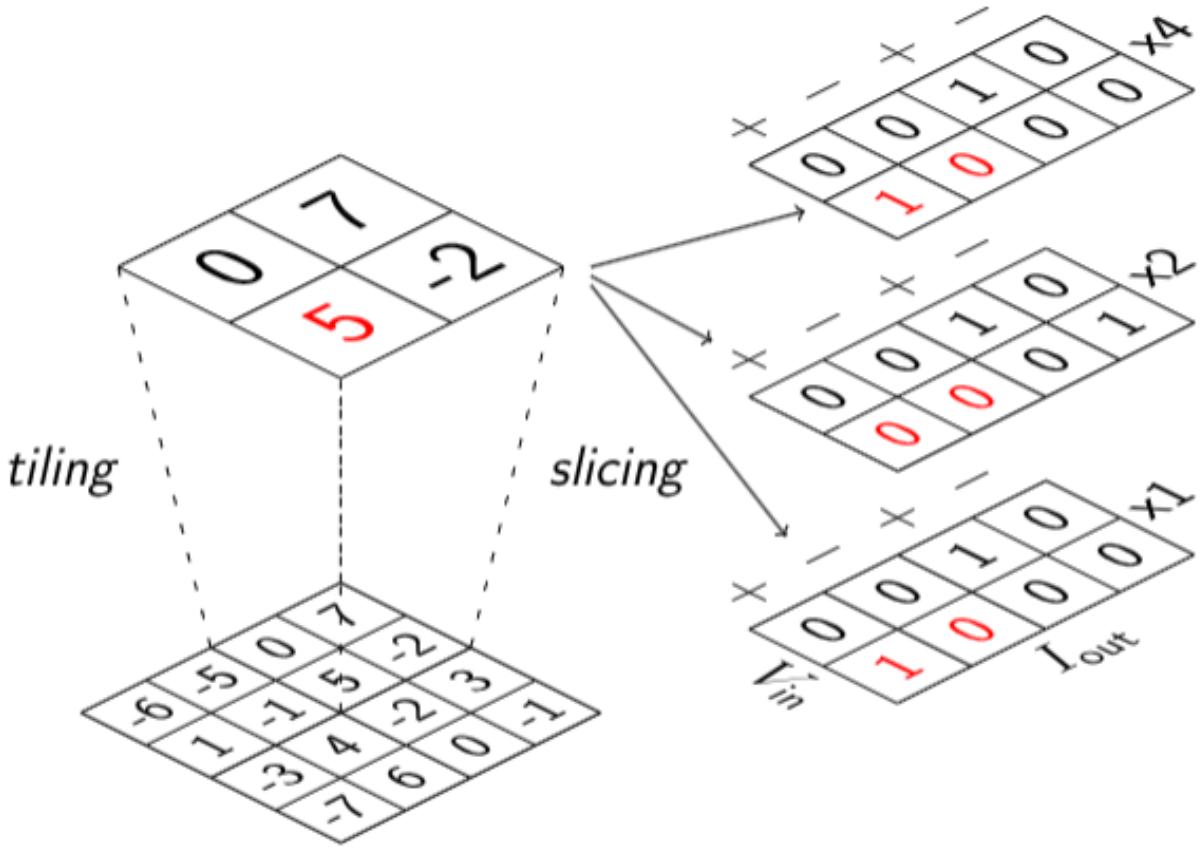


Figure: Mapping a weight matrix on memristor arrays [Rossenbach^{+} 25]

FFNN Transducer: 8-bit vs. 4-bit Quantization

Table: WER (%; memristor mean ± standard deviation) on LibriSpeech dev-other.

	Modules Quantized			Lex-Free	Tree	
Weight bits	**Encoder**	**Predictor**	**Joiner**	**No LM**	**No LM**	**4-gram**
Baseline						
–	–	–	–	5.9	5.8	4.8
QAT						
8	✓	–	–	6.3	6.1	4.9
8	✓	✓	–	6.4	6.2	5.0
8	✓	✓	✓	6.4	6.2	4.9
4	✓	–	–	7.2	6.8	5.2
4	✓	✓	–	7.3	6.9	TBD
4	✓	✓	✓	TBD	TBD	TBD
Memristor Simulation						
8	✓	–	–	9.8 ± 0.06	8.7 ± 0.03	6.7 ± 0.07
8	✓	✓	–	10.7 ± 0.10	8.9 ± 0.11	6.7 ± 0.04
8	✓	✓	✓	44.7 ± 1.95	56.1 ± 10.90	18.8 ± 1.24
4	✓	–	–	10.1 ± 0.04	9.1 ± 0.12	6.6 ± 0.07
4	✓	✓	–	10.4 ± 0.11	9.3 ± 0.12	6.7 ± 0.06
4	✓	✓	✓	37.4 ± 1.33	29.6 ± 2.39	12.8 ± 0.72

Run counts not supplied in the aggregate report. TBD: QAT-only scores not supplied.

Full QAT FFNN Transducer Deep Dive

Table: WER (%; mean ± standard deviation) on LibriSpeech dev-other. The entire 8-bit model is trained under QAT.

Memristor Simulated Modules			Lex-Free	Tree	
Encoder	**Predictor**	**Joiner**	**No LM**	**No LM**	**4-gram**
✓	✓	✓	44.7 ± 1.95	56.1 ± 10.90	18.8 ± 1.24
✓	–	✓	45.4 ± 4.99	55.1 ± 6.60	21.8 ± 4.84
✓	✓	–	9.8 ± 0.03	8.7 ± 0.10	6.7 ± 0.03
✓	✓	1st Layer Only	11.2 ± 0.18	9.4 ± 0.09	6.8 ± 0.05

Updated aggregates; run counts are not supplied in the report.

All rows except the first row could possibly be tuned to improve WER.

Device VMM Operation Statistics

operation	average	std-dev	min	max
1.0×1	1.016	0.063	0.707	1.27
0.1×1	$0.989 \cdot 10^{-1}$	$0.062 \cdot 10^{-1}$	$0.688 \cdot 10^{-1}$	$1.24 \cdot 10^{-1}$
0.01×1	$0.985 \cdot 10^{-2}$	$0.069 \cdot 10^{-2}$	$0.721 \cdot 10^{-2}$	$1.20 \cdot 10^{-2}$
1.0×0	$8.52 \cdot 10^{-4}$	$4.75 \cdot 10^{-2}$	-0.2751	0.2455
1.0×0.5	0.476	0.094	0.109	1.06

Table: Hardware measurement statistics for simulated memristor VMM operations [Rossenbach^{+} 25].